

Unit - 8 : Input and Output

Topic : Introduction, Data Streams

1. Introduction

- Input and Output (I/O) computer system का एक important part है जो program को user, files और other devices के साथ communicate करने की सुविधा देता है।
- Program को data प्राप्त करने के लिए Input की आवश्यकता होती है और result को बाहर भेजने के लिए Output की।
- Python में Input and Output operations को handle करने के लिए built-in functions और I/O classes का use किया जाता है।
- I/O operations का use file processing, user interaction, device communication और data transfer के लिए होता है।
- Python में सभी I/O operations "streams" के रूप में handle किए जाते हैं, जिससे data को एक sequence (flow) में पढ़ा या लिखा जा सकता है।

Note : Input → program के लिए data प्राप्त करना
Output → program द्वारा result बाहर भेजना

2. Need of Input and Output

- User से data लेने के लिए (जैसे keyboard input)
- Program का result दिखाने के लिए (जैसे screen output)
- File में data store करने और पढ़ने के लिए
- Other devices (जैसे printer, network, sensors) के साथ communication के लिए
- Data को permanently storage में रखने के लिए

3. Input - Output Process

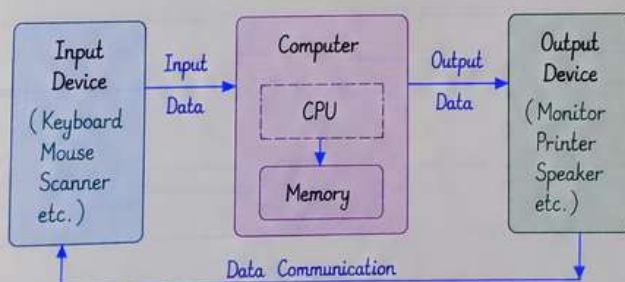


Fig : Input - Processing - Output

4. Standard Input, Output and Error Streams

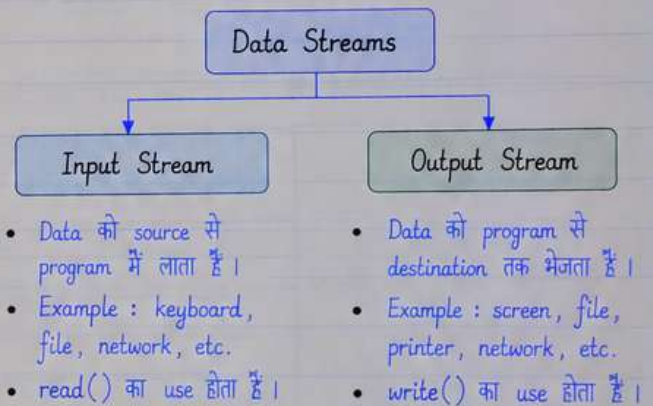
- Python में कुछ pre-defined standard streams होती हैं :
 - ① `sys.stdin` → standard input stream (keyboard input)
 - ② `sys.stdout` → standard output stream (screen output)
 - ③ `sys.stderr` → standard error stream (error messages)
- ये streams program को input लेने, output देने और error messages दिखाने के लिए automatically available होती हैं।

5. What are Data Streams?

- Data Stream एक continuous flow of data होता है, जिसके द्वारा data sequential form में एक source से दूसरे destination तक भेजा या प्राप्त किया जाता है।
- Stream को एक "flow of bytes" के रूप में देखा जाता है।
- Python में input और output दोनों stream के रूप में handle किए जाते हैं।
- Stream से data एक-एक करके (sequentially) read या write किया जाता है।
- Data Stream file, device, network connection या memory से हो सकता है।

Data Stream = Sequence of data (bytes/characters) flowing from source to destination.

6. Types of Data Streams



7. Stream Operations

Operation	Description	Common Methods
Open	Stream को open करना	<code>open()</code>
Read	Stream से data पढ़ना	<code>read()</code> , <code>readline()</code> , <code>readlines()</code>
Write	Stream में data लिखना	<code>write()</code> , <code>writelines()</code>
Close	Stream को close करना	<code>close()</code>
Flush	Buffer का data तुरंत write करना	<code>flush()</code>

8. Key Points

- Input और Output operations Python में streams के रूप में work करते हैं।
- Data Stream एक continuous flow of data होता है।
- Standard streams (`stdin`, `stdout`, `stderr`) हमेशा available रहती हैं।
- Streams के माध्यम से हम files, devices और network के साथ easily communicate कर सकते हैं।
- Proper stream handling से program अधिक efficient और error-free बनता है।

Topic : Creating Your Own Data Streams , Access Modes

1. Creating Your Own Data Streams

- Python में हम अपने program के लिए user-defined data streams create कर सकते हैं, ताकि हम files, devices या other sources से data read और write कर सकें।
- जब हम कोई file open करते हैं, तो Python एक file object (create) करता है, जो एक data stream की तरह काम करता है।
- इस data stream के माध्यम से हम data को sequentially read या write कर सकते हैं।
- यह हमें program में external data (files) को handle करने की सुविधा देता है।

Creating a Data Stream (File)



2. Steps to Create and Use a Data Stream

- ① Open the file using open() function.
- ② Perform read() or write() operations.
- ③ Process the data as required.
- ④ Close the file using close() function (or use with statement).
- ⑤ Handle exceptions (if any).

3. Example : Creating and Using a Data Stream

```
# Creating your own data stream (file)
try:
    # Open a file in write mode
    f = open("sample.txt", "w")
    f.write("Hello! This is my own data stream.\n")
    f.write("Python makes it easy to work with files.\n")
    f.close()
    print("Data written successfully.")
except Exception as e:
    print("Error :", e)

try:
    # Open the same file in read mode
    f = open("sample.txt", "r")
    print("\nFile Content:")
    data = f.read()
    print(data)
    f.close()
except Exception as e:
    print("Error :", e)
```

Output :

Data written successfully.
File Content :
Hello! This is my own data stream.
Python makes it easy to work with files.

4. Access Modes

- Access mode यह decide करता है कि file को किस purpose के लिए open किया जा रहा है, जैसे read करने के लिए, write करने के लिए या दोनों के लिए।
- Python में open() function के साथ access mode specify किया जाता है।
- हर mode का अपना specific काम होता है।
- अगर file exist नहीं करती है और हम write mode में open करते हैं, तो Python नई file create कर देता है।

5. Common Access Modes

Mode	Full Form	Description
r	Read	File को read करने के लिए open करता है। (File exist होना चाहिए)
w	Write	File को write करने के लिए open करता है। (अगर file exist नहीं है तो नई file create हो जाती है)
a	Append	File के end में data जोड़ने के लिए open करता है। (File exist नहीं है तो नई file create होती है)
r+	Read and Write	File को read और write दोनों के लिए open करता है। (File exist होना चाहिए)
w+	Write and Read	File को write और read दोनों के लिए open करता है। (नई file create हो जाती है)
a+	Append and Read	File में data append और read दोनों के लिए open करता है। (File exist नहीं है तो नई file create होती है)

6. Example : Different Access Modes

<pre># Write Mode (w) f = open("demo.txt", "w") f.write("This is write mode.\n") f.close()</pre>	<pre># Append Mode (a) f = open("demo.txt", "a") f.write("This is append mode.\n") f.close()</pre>
<pre># Read Mode (r) f = open("demo.txt", "r") print(f.read()) f.close()</pre>	<pre># Read and Write Mode (r+) f = open("demo.txt", "r+") f.write("Using r+ mode.\n") f.seek(0) # move to start print(f.read()) f.close()</pre>

7. Key Points

- Data stream एक continuous flow of data होता है।
- File handling के लिए open() function का use किया जाता है।
- Access mode यह तय करता है कि file को किस operation के लिए open करना है।
- Always close the file after use (या with statement का use करें)।
- Invalid mode या गलत operation करने पर error आ सकता है।
- Proper access mode का use करने से data loss से बचा जा सकता है।

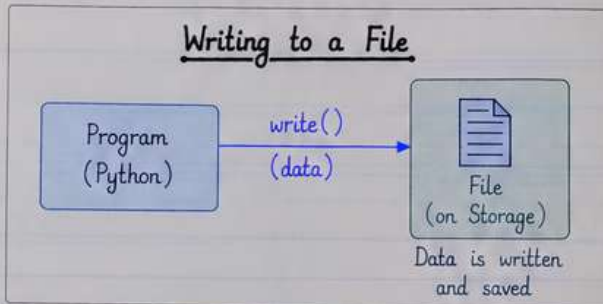
Conclusion :

Python में अपने data streams बनाना और सही access mode का use करना file handling को आसान, flexible और efficient बनाता है।

Topic : Writing Data to a File • Reading Data From a File

1. Writing Data to a File (Introduction)

- File में data write करने के लिए हम `open()` function का use करते हैं।
- File को write mode ("w"), append mode ("a") आदि में open किया जा सकता है।
- `write()` function द्वारा string या data file में लिखा जाता है।
- अगर file exist नहीं करती है, तो write mode में open करने पर नई file create हो जाती है।
- File में data लिखने के बाद file को `close()` करना जरूरी होता है ताकि data properly save हो जाए।



2. Syntax

```
f = open("filename.txt", "w") # open in write mode
f.write("data...")           # write data
f.close()                    # close the file
```

3. Example : Writing Data to a File

```
# Example : write some text to a file
f = open("sample.txt", "w")
f.write("Hello!\n")
f.write("This is written to a file.\n")
f.write("Python is easy to learn.")
f.close()
print("Data written successfully.")
```

4. Modes for Writing

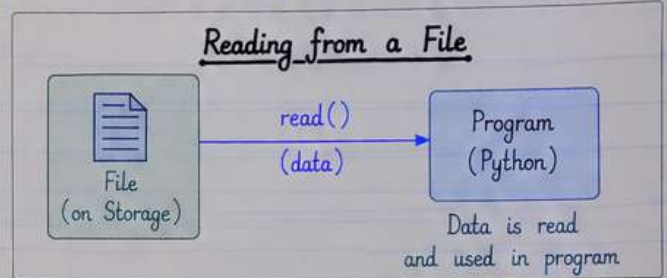
Mode	Description
"w"	Write mode (old data overwrite)
"a"	Append mode (data end में add होता है)
"x"	Create new file (अगर file exist नहीं करती है)
"w+"	Read and write (overwrite)
"a+"	Read and write (append)

Note :

- `write()` के लिए data string के रूप में होता चाहिए।
- अगर हमें numbers लिखने हैं, तो उन्हें string में convert करना होगा (जैसे `str(10)`)।
- File close करना हमेशा अच्छी practice है।

5. Reading Data From a File (Introduction)

- File से data read करने के लिए हम `open()` function का use read mode ("r") में करते हैं।
- `read()` function file का पूरा content एक string के रूप में return करता है।
- `readline()` function file की एक line पढ़ता है।
- `readlines()` function file की सभी lines को list के रूप में return करता है।
- अगर file exist नहीं करती है, तो `FileNotFoundError` मिलता है, इसलिए error handling करना अच्छी practice है।



6. Syntax

```
f = open("filename.txt", "r") # open in read mode
data = f.read()              # read all data
f.close()                    # close the file
```

7. Example : Reading Data from a File

```
# Example : read the content of a file
f = open("sample.txt", "r")
data = f.read()
print("File Content :")
print(data)
f.close()
```

8. Other Methods for Reading

Method	Description
<code>read()</code>	पूरा file content पढ़ता है (string के रूप में)
<code>readline()</code>	एक line पढ़ता है
<code>readlines()</code>	सभी lines को list में return करता है
<code>for line in f</code>	file की line by line पढ़ने के लिए use होता है

9. Example : Read Line by Line

```
f = open("sample.txt", "r")
print("Reading line by line:")
for line in f:
    print(line.strip()) # strip() नए line हटाता है
f.close()
```

Key Points :

- File handling से data को permanent storage में save और retrieve किया जा सकता है।
- Proper mode का use करें (write/read/append)।
- File operations के दौरान error handling जरूर करें।
- काम पूरा होने के बाद file को `close()` करना न भूलें।

Topic : Additional File Methods, Using Pipes as Data Streams

1. Additional File Methods (Introduction)

- Python file objects provide several useful methods jo file handling ko aur flexible बनाते हैं।
- ये methods read, write, file position, status check और file management के लिए use होते हैं।
- इन methods का use करके हम file के साथ better control और efficient data processing कर सकते हैं।

2. Common Additional File Methods

Method	Description
f.read(size)	size bytes का data read करता है। (यदि size नहीं दिया तो पूरा file read करेगा)
f.write(data)	data को file में लिखता है।
f.readline()	file की एक line read करता है।
f.readlines()	file की सभी lines को list के रूप में return करता है।
f.seek(offset)	file pointer को specified position पर ले जाता है।
f.tell()	current position (file pointer) को return करता है।
f.flush()	buffer का data तुरंत file में लिखता है।
f.close()	file को close करता है।
f.readable()	check करता है कि file readable है या नहीं।
f.writable()	check करता है कि file writable है या नहीं।
f.seekable()	check करता है कि file seek (reposition) किया जा सकता है या नहीं।

3. Example : Using Additional File Methods

```
# Example to demonstrate additional file methods
f = open("example.txt", "r+")
print("Is file readable?", f.readable())
print("Is file writable?", f.writable())
print("Current position :", f.tell())
line = f.readline()
print("First line :", line)
f.seek(0) # move to beginning
print("After seek, position :", f.tell())
f.close()
```

4. Key Points (File Methods)

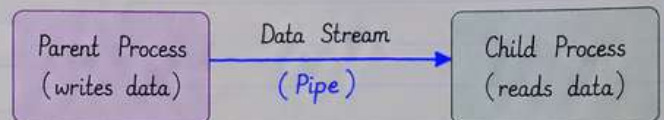
- seek() का use file pointer ko move karne के लिए होता है।
- tell() current position प्राप्त करने के लिए use होता है।
- readline() और readlines() sequential reading के लिए useful हैं।
- flush() का use buffer का data तुरंत save करने के लिए होता है।
- File को close() करना हमेशा good practice है।

5. Using Pipes as Data Streams (Introduction)

- Pipe एक communication mechanism है जिसका use दो processes के बीच data transfer करने के लिए होता है।
- Python में pipes को data streams के रूप में use किया जा सकता है।
- Pipes के माध्यम से एक program दूसरे program को data भेजता है और दूसरा program उस data को read कर सकता है।
- यह technique inter-process communication (IPC) के लिए बहुत useful होती है।

6. How Pipes Work ?

- एक process (parent) pipe में data लिखता है।
- दूसरा process (child) उसी pipe से data पढ़ता है।
- Pipe एक unidirectional data flow provide करता है (आमतौर पर parent → child)।
- Python में os module या subprocess module का use करके pipes create किए जा सकते हैं।



7. Example : Using Pipe with os Module

```
import os
# Create a pipe
read_fd, write_fd = os.pipe()
pid = os.fork()
if pid > 0:
    # Parent process
    message = "Hello from parent!"
    os.write(write_fd, message.encode())
    os.close(write_fd)
else:
    # Child process
    os.close(write_fd)
    data = os.read(read_fd, 1024)
    print("Child received :", data.decode())
    os.close(read_fd)
```

8. Key Points (Pipes)

- Pipe ek simple और efficient way है दो processes के बीच data transfer करने का।
- os.pipe() दो file descriptors (read और write) return करता है।
- os.fork() द्वारा child process create किया जाता है।
- Parent process pipe में data लिखता है और child process उस data को read करता है।
- Pipes का use large data transfer और real-time communication में भी किया जाता है।

Topic : Handling IO Exceptions

1. Introduction

- File handling करते समय कई बार Input/Output (I/O) related errors आ सकते हैं, जैसे file न मिलना, permission का न होना, disk full होना आदि।
- इन परिस्थितियों में program terminate हो सकता है, इसलिए I/O operations में exception handling करना बहुत जरूरी है।
- Python में I/O errors सामान्यतः OSError class के under आते हैं, जिसके कई specific subclasses होती हैं (जैसे FileNotFoundError)
- Try-except block का use करके इन exceptions को handle किया जाता है ताकि program crash न हो और user को meaningful error message मिल सके।

2. Common IO Exceptions

Exception	When it occurs	Example Situation
FileNotFoundError	जब specified file exist नहीं करती	open("data.txt")
PermissionError	जब file को access करने की permission नहीं होती	read/write protected file
IsADirectoryError	जब file की जगह directory को open करने की कोशिश होती है	open("folder/", "r")
NotADirectoryError	जब directory की जगह file को directory की तरह use किया जाता है	open("file.txt/abc")
OSError	General I/O related error	Disk error, device not available etc.

3. Basic Example : Handling IO Exception

Example: reading a file with exception handling
try:

```
f = open("data.txt", "r")
content = f.read()
print("File Content: ")
print(content)
f.close()
```

```
except FileNotFoundError as e:
    print("Error: File not found.", e)
except PermissionError as e:
    print("Error: Permission denied.", e)
except OSError as e:
    print("General I/O error:", e)
```

Output (if file not found):

Error: File not found. [Errno 2] No such file or directory: 'data.txt'

4. Using Multiple except Blocks

- Different I/O exceptions को अलग-अलग except blocks में handle करना best practice है।
- इससे हमें specific error का पता चलता है और उचित action लेना आसान होता है।
- अंत में general exception (OSError) को भी handle करना चाहिए ताकि कोई unexpected error न छूटे।

5. Example : Writing to a File

Example: writing data to a file with exception handling
try:

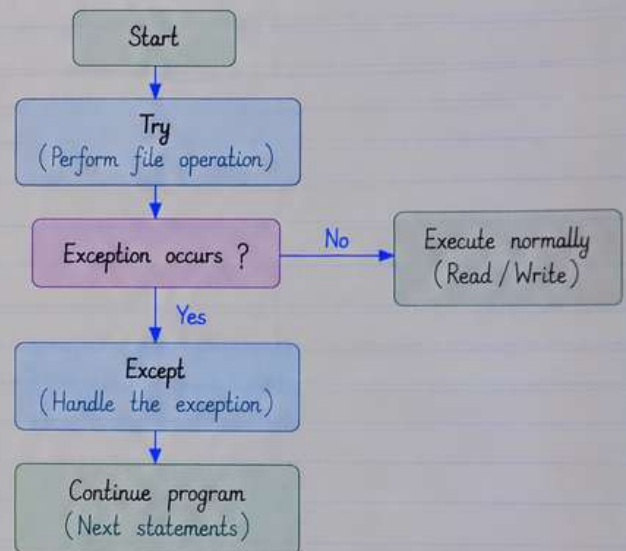
```
f = open("output.txt", "w")
f.write("Hello Python!\n")
f.write("File handling with exceptions.\n")
f.close()
print("Data written successfully.")
```

```
except PermissionError:
    print("Error: You don't have permission to write this file.")
except OSError as e:
    print("I/O Error:", e)
```

Output (on success):

Data written successfully.

6. Flow of Exception Handling in File I/O



7. Best Practices

- File operations हमेशा try-except block के अंदर करें।
- Specific exceptions (जैसे FileNotFoundError, PermissionError) को separately handle करें।
- User को clear और meaningful error message दें।
- File को use करने के बाद हमेशा close() करें (या with statement का use करें)।
- Unexpected I/O errors के लिए general OSError exception को handle करना न भूलें।
- Proper exception handling से program reliable और user-friendly बनता है।

8. Key Takeaways

- I/O operations में errors आना सामान्य है।
- Python इन errors को exceptions के रूप में handle करता है।
- Try-except block program को crash होने से बचाता है।
- सही exception handling से program robust और maintainable बनता है।

Topic : 3 examples of Input and Output

- Input and Output (I/O) program को user के साथ interact करने की सुविधा देता है।
- Input के माध्यम से हम data (जैसे number, string आदि) user से प्राप्त करते हैं।
- Output के माध्यम से हम result को screen पर display करते हैं या file में write करते हैं।
- Python में input() function और print() function I/O के लिए सबसे common functions हैं।

1. Example 1 : Taking Input from User and Displaying Output

Problem :

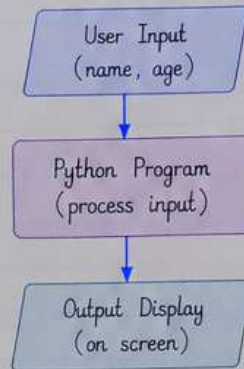
User से साफ़का नाम और उम्र (age) input लेकर उसे output में display करना।

Code :

```
# Taking input from user
name = input("Enter your name : ")
age = input("Enter your age : ")

# Displaying output
print("\nHello", name, "!")
print("You are", age, "years old.")
```

Flow / Working :



Sample Input :

Enter your name : Rahul
Enter your age : 20

Sample Output :

Hello Rahul !
You are 20 years old.

Explanation :

- input() function से user का data string के रूप में प्राप्त होता है।
- print() function उस data को output के रूप में display करता है।

2. Example 2 : Reading Data from a File and Displaying Output

Problem :

एक text file से data पढ़कर उसे screen पर display करता।

Code :

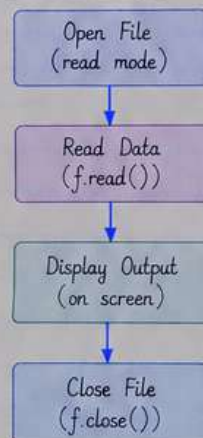
```
# Open the file in read mode
f = open("sample.txt", "r")

# Read the content
data = f.read()

# Display the content
print("File Content : ")
print(data)

# Close the file
f.close()
```

Flow / Working :



Sample File (sample.txt) :

Python is easy to learn.
Input and Output makes
programs interactive.
Keep Practicing !

Sample Output :

File Content :
Python is easy to learn.
Input and Output makes
programs interactive.
Keep Practicing !

Explanation :

- open() function file को read mode में open करता है।
- read() function से पूरा data एक string के रूप में प्राप्त होता है।
- File को use करने के बाद हमेशा close() करना चाहिए।

3. Example 3 : Writing Data to a File (with User Input)

Problem :

User से एक message input लेकर उसे file में write करना और success message display करना।

Code :

```
# Taking input from user
msg = input("Enter a message to save : ")

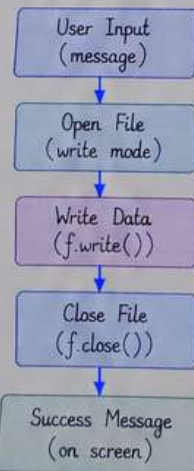
# Open the file in write mode
f = open("message.txt", "w")

# Write the message
f.write(msg)

# Close the file
f.close()

print("Message saved successfully !")
```

Flow / Working :



Sample Input :

Enter a message to save : I love Python!

File Content (message.txt) :

I love Python!

Sample Output :

Message saved successfully !

Explanation :

- input() से user का message प्राप्त किया जाता है।
- write() function उस message को file में लिखता है।
- File को close() करना जरूरी है ताकि data properly save हो सके।
- यह method log file, notes या user data store करने के लिए useful होता है।

Key Point : Input और Output की मदद से program user, files और devices के साथ communicate कर सकता है।